



KickStart Streaming

- ☐ Enterprise Integration Patterns by Gregor Hohpe
 - File, DB, RPC, Messaging.
- ☐ Messaging: Reason for messaging/Broker: Point-to-point mess. (Airport Hub example)
 - Broker Producer, Consumer.
 - Types of use case: low latency. (Async)
 - State-based systems: request-response or Request Driven systems
 - Event-Driven Systems, Document migration, Analytical Dashboard
 - 3 types of Message: Event, Command, Document.
- ☐ Timeline of Confluent's platform components
 - Apache Kafka: 2012; 2015:SR, connect, RP ;2016:iconnect, kafkastreams;2018:ksql; 2020:ksqlDB;2022:KRaft;2024:cp-flink; 2025: tableflow
- ☐ DSE: Streaming Platform:
 - STREAM (Kafka). PROCESS (Flink, kStreams). CONNECT(Kafka Connect) Govern (SR)
- ☐ Deployment Topology:
 - Java application forming a cluster, distributed , JVM deployment.
- ☐ DSE Exam: 30 Q, 1 hour, 24 is pass(80%), Kafka :14, Flink 8 , SR:2, Connect:3, KStreams: 3

Apache Kafka

- ☐ - Distributed , commit log, key value, Elastic, Scalable (immutable commit log)
 - Broker, Producer, consumer (JVM), Zookeeper/KRaft
 - Q : deductive read
 - Apache kafka protocol
- ☐ Apache kafka Record (Topic, partition*,Timestamp*,Headers* K*,V)
- ☐ Broker
 - Reliable & Durable storing of AK records.
 - Store Data and process requests from Producer, consumer, other brokers
 - Metadata management, locally caching Metadata from ZK.Controller
 - Single Active controller.
 - Leader Election for partition, manager & propagates metadata to all other broker
- ☐ Zookeeper or KRaft -> Distributed coordination and management service
 - Broker Registration & cluster membership
 - Controller election
 - Metadata mangement: Topics,Partition, replicas,preferred leaders,ACL, quotas, config details
- ☐ Topics
 - Regular topic , compacted topic
- ☐ Partitions
 - Need for partitioning
 - Hashmap partitioning example
- ☐ Replication
 - RF, ISR
- ☐ Producer
 - Producer internals: serializer, partitioner, batching, compression, Acks
 - idempotent producer, batching(inger...
 - Availability, throughput, latency, durability
- ☐ Consumer
 - consumer grp, regex
 - __consumer_offsets topic, assign() vs subscribe() , auto commit by default
 - poll vs Fetch
- ☐ DLQ

SR

- ☐ Serializable/Encoding/Decoding.
 - InMemory, Over the network send,
 - Locked(Java & pickle python). Cross language. XML, JSON, BSON
 - Schema on Read, Schema on Write.
- ☐ - Constructs vs constrains (Message:kv; Topic: Many message; Schema (or event type):Structure of message(kv);Subject:schema,v1)
 - Constraints: Key and value has different schemas.
 - Schema:message (message has a structure)
 - subject-schema (schema,v1 or v2, depends on the compatibility level of schema)
 - subject-topic (default NamingStrategy: topicnamingstrategy, topic-key, topic-value, Colour coding the pipes)
 - other Strategy: recordnaming(customershippingupdate,customeraddressupdate), topicrecordnaming
- ☐ AVRO, JSON, PROTOBUF, Data Contracts
- ☐ Compatibility
 - Backward, Forward, Full, Transitive
- ☐ Governance: Stream Catalog, Stream Quality, Stream Lineage

Connect

- ☐ JVM, Worker, Tasks (tasks parallelism)
- ☐ Source Connector, Sink Connector, converter, SMTs
- ☐ Connect Hub, Atleast once connector, Exactly once.
- ☐ Sample left to right

Stream Concepts

- ☐ Stream, river, pond(Dam)
- ☐ Batch Processing Vs Stream Stream Processing
- ☐ Stream, State, Time, Snapshot
 - Backpressure handling
- ☐ Directed Acyclic Graphs(DAGs)
- ☐ Stream Table Duality
- ☐ Gotchas: Stream Repartitioning , shuffling (Group By, Sum,min, count)

Kafka Streams

- ☐ Consume, process, produce pattern
- ☐ Serdes
- ☐ State (RocksDB, changelogStream to kafka topics).
- ☐ Table, KTable, GlobalKTable (Stream/Table Duality)
- ☐ KafkaStreams Deployment

Apache Flink

- ☐ Deployment, Batching as Specialized form of Streaming
- ☐ WaterMarking,SavePoint, Checkpoint
- ☐ StreamTable Duality: AppendOnly Table, Update Table
- ☐ API: DataStream, table, SQL
- ☐ INSERT INTO results SELECT color, COUNT(*) FROM EVENTS where color <> orange GROUP BY color;

differences

- ☐ Deployment
- ☐ Watermarking
- ☐ StateStore
- ☐ ExactlyOnce/ Transactions